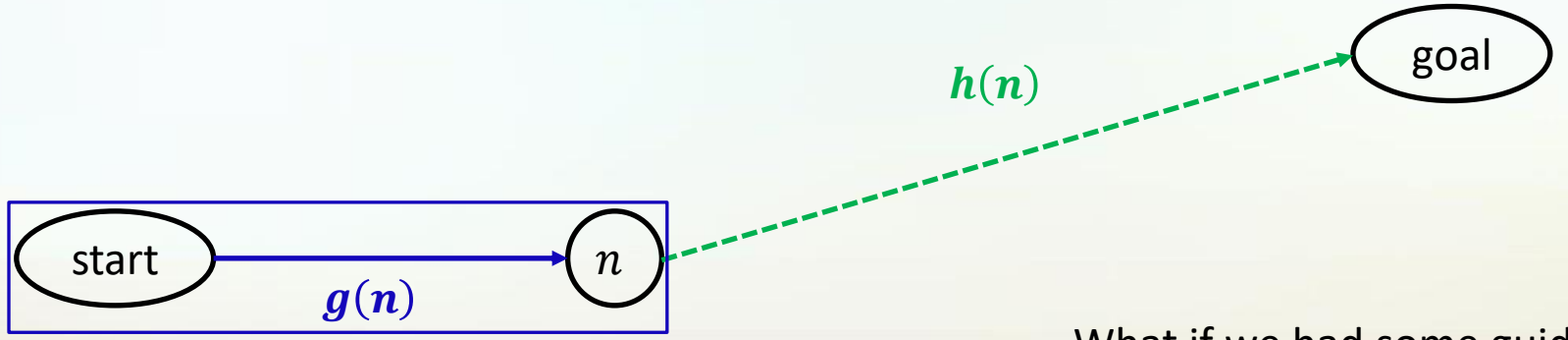
Informed Search

Uniformed Search vs. Informed Search

What we know with uninformed search:

- $\boldsymbol{g(n)}$: path cost from start to state n.
- Successor states



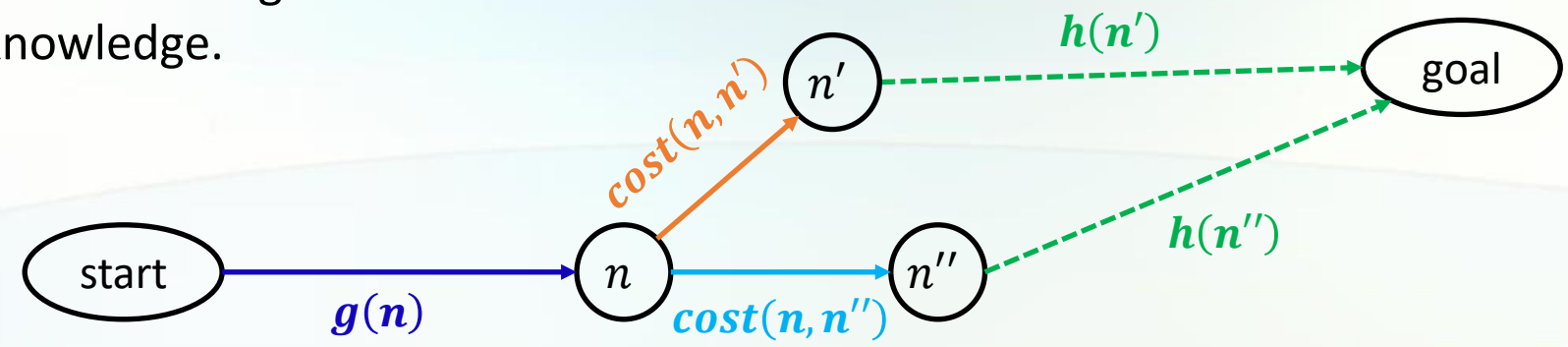
What if we had some guidance about which direction to search?

With ***Informed search***, we know:

- All properties from uninformed search
- $\boldsymbol{h(n)}$: a heuristic path cost from n to the goal.

Using the Heuristic

Goal: speed up search using domain knowledge.



Recall with uniform-cost search:

- We store potential next states with a priority queue
- Expand the state with the smallest "first-half-cost": $g(n)$

$h(n)$ is a "second-half-cost"

How can we use $h(n)$?

A Classic Example

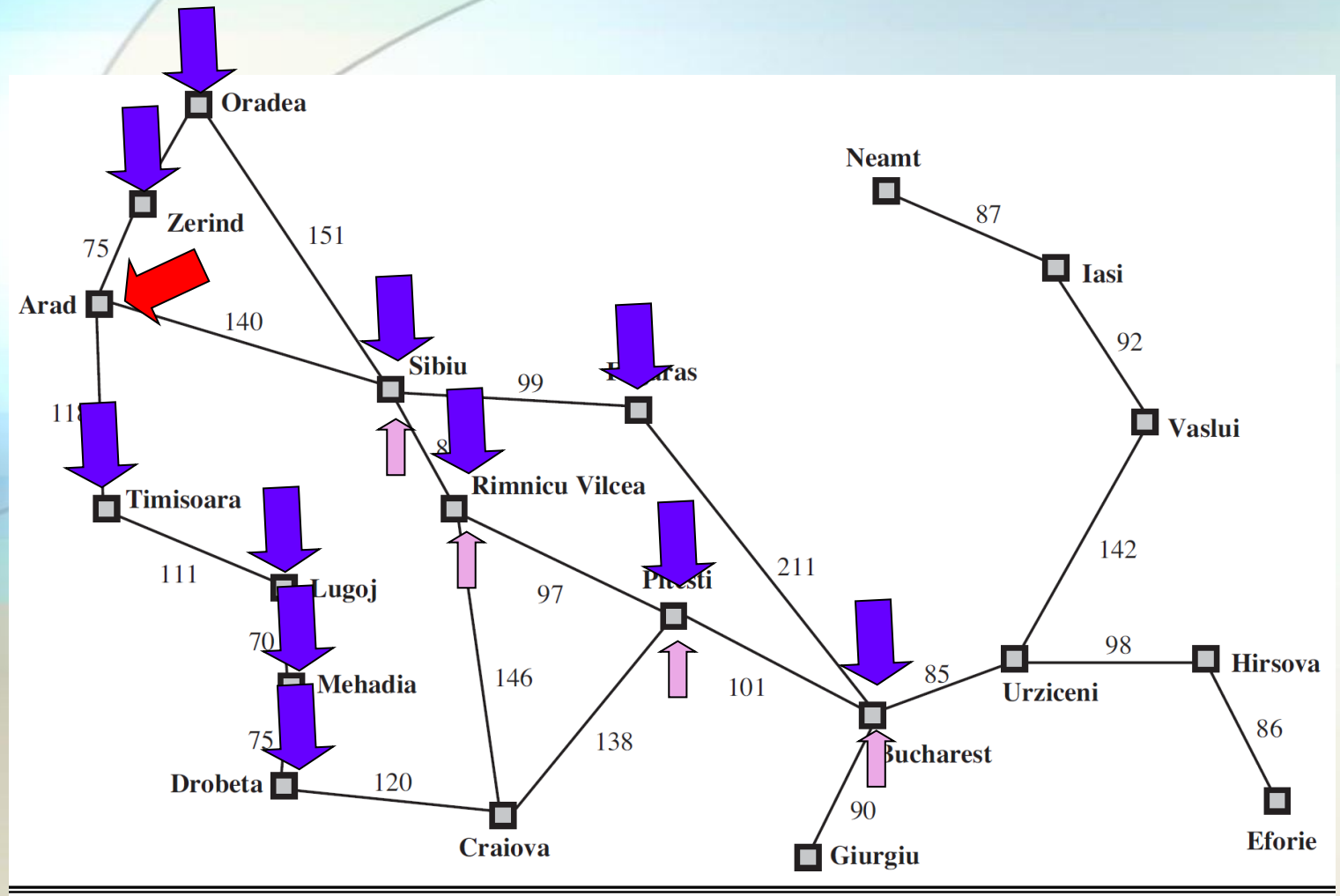


Figure 3.2 A simplified road map of part of Romania.

Straight-line Distances to Bucharest

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

How does UCS proceed?

Recall, it minimizes $g(n)$.

Problem: Visit too many nodes, some ***clearly*** out of the question.

Visualizing UCS vs. A*

Attempt: Greedy Best-First

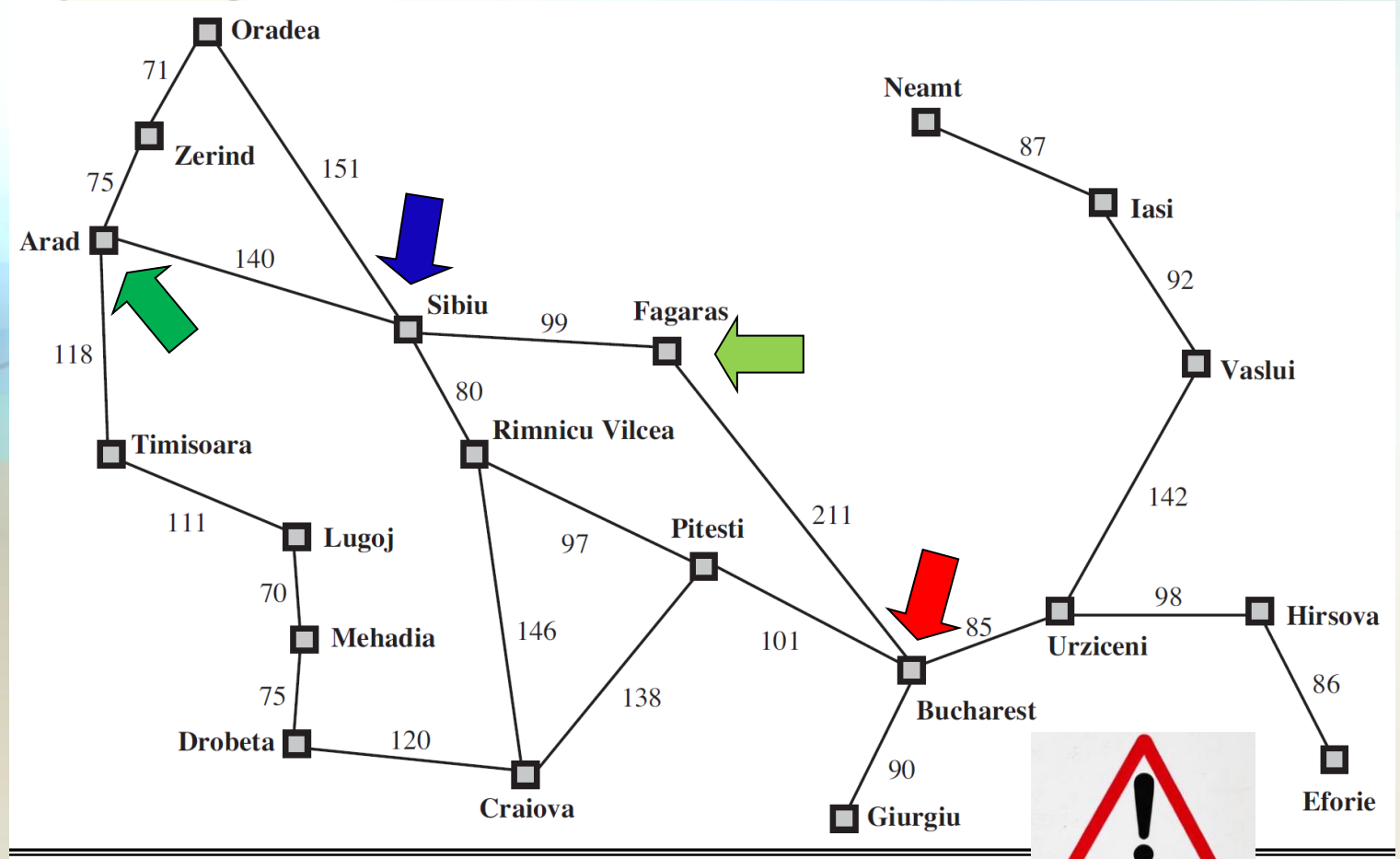


Figure 3.2 A simplified road map of part of Romania.

Approach: Choose smallest $h(n)$

Straight-line Distances to Bucharest

Arad	366	Mehadia	241
Bucharest	0	Neamt	234
Craiova	160	Oradea	380
Drobeta	242	Pitesti	100
Eforie	161	Rimnicu Vilcea	193
Fagaras	176	Sibiu	253
Giurgiu	77	Timisoara	329
Hirsova	151	Urziceni	80
Iasi	226	Vaslui	199
Lugoj	244	Zerind	374

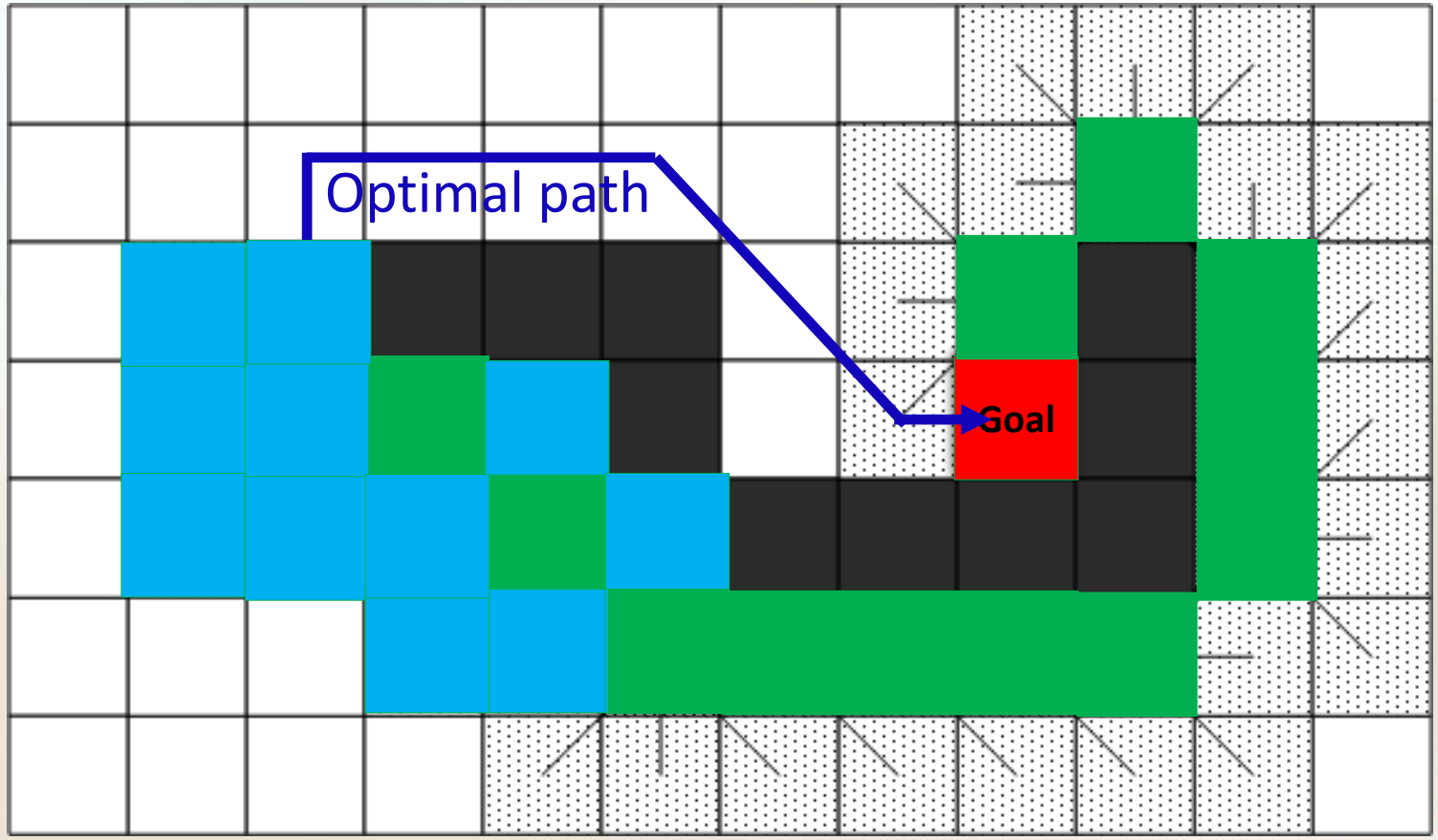
Arad → Sibiu → Rimnicu Vilcea → Pitesti

Greedy Best-First

Complete? ✓ *assuming no cycles

Optimal? ✗

Explore the grid expanding node with shortest distance to goal.



Issue: Greedy Best-First ignores path gets lured by promising $h(n)$ heuristic values and does not account for cost to reach those nodes: $g(n)$.

A* Search By Example

Approach: Expand smallest $g(n) + h(n)$

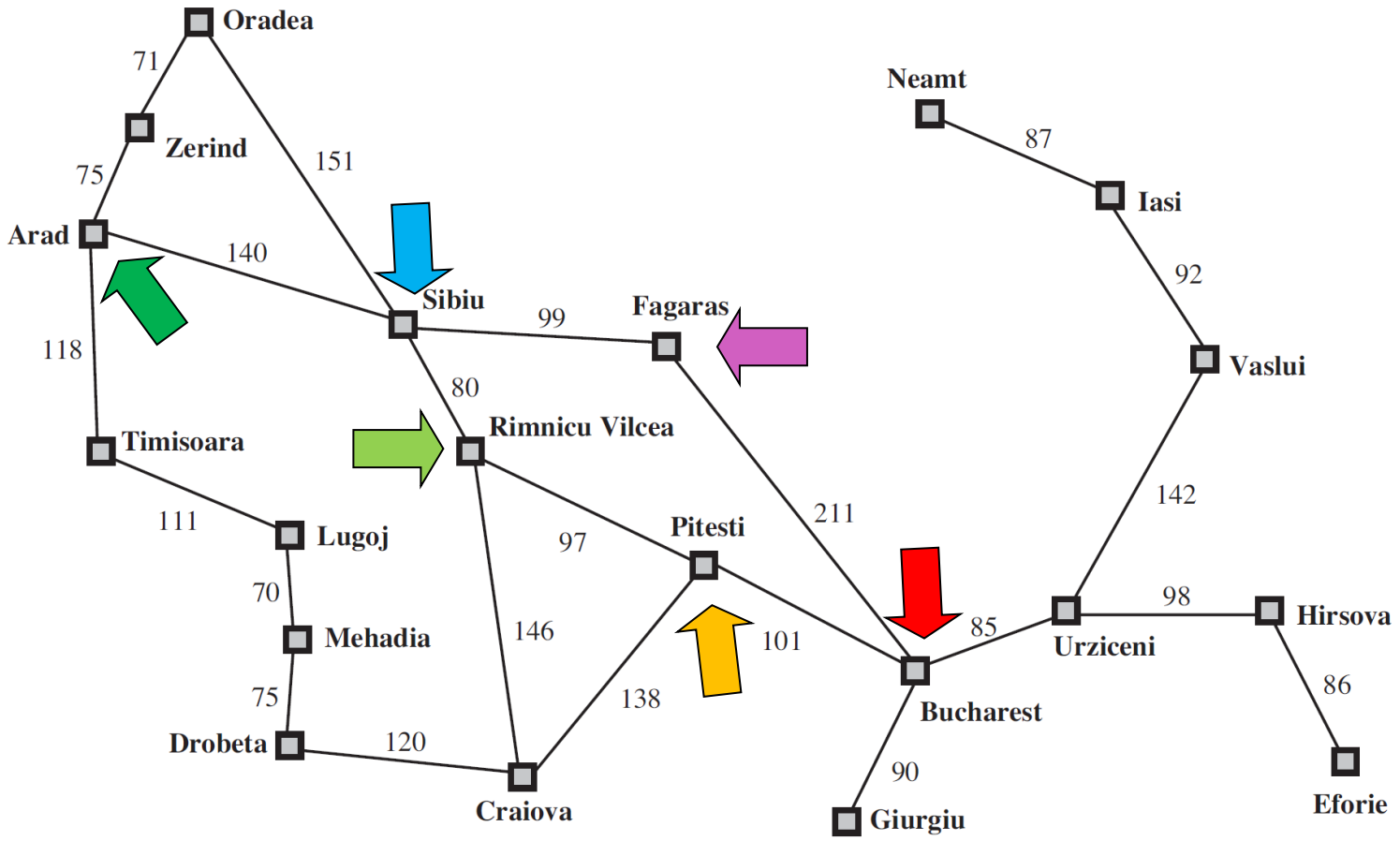


Figure 3.2 A simplified road map of part of Romania.

Straight-line Distances to Bucharest

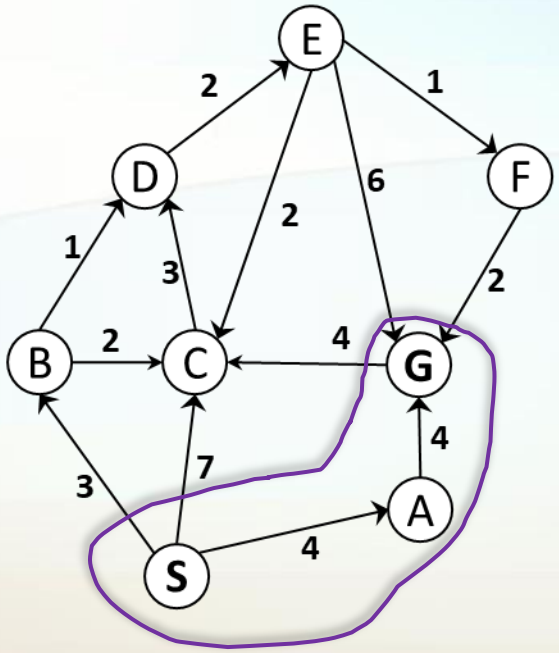
Arad	366	
Bucharest	0	$450 + 0 = 450$
Craiova	160	$366 + 160 = 526$
Drobeta	242	
Eforie	161	
Fagaras	176	$239 + 176 = 415$
Giurgiu	77	
Hirsova	151	
Iasi	226	
Lugoj	244	
Mehadia	241	
Neamt	234	
Oradea	380	
Pitesti	100	$317 + 100 = 417$
Rimnicu Vilcea	193	$220 + 193 = 413$
Sibiu	253	$140 + 253 = 393$
Timisoara	329	$118 + 329 = 447$
Urziceni	80	
Vaslui	199	
Zerind	374	$75 + 374 = 449$

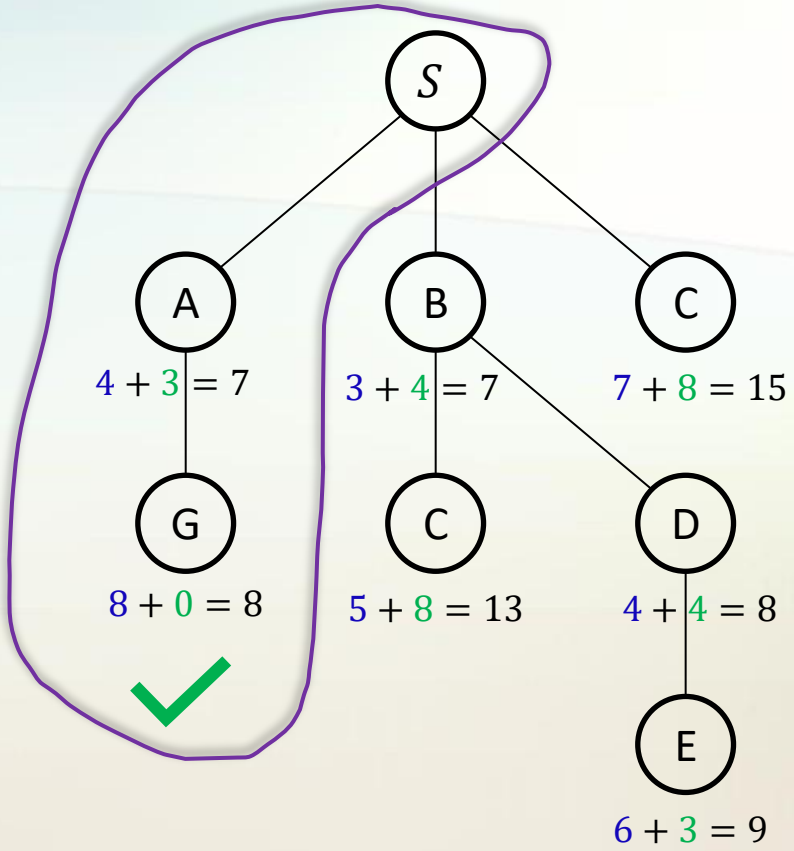
$418 + 0 = 418$

$455 + 160 = 615$

Quick Example

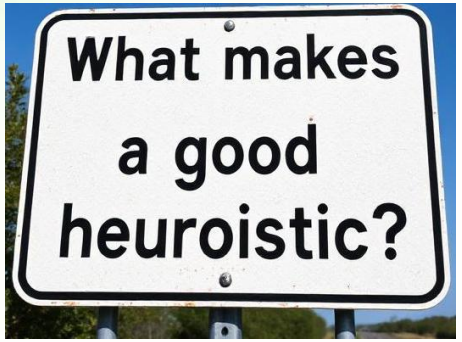
Node	$h(x)$
S	7
A	3
B	4
C	8
D	4
E	3
F	2
G	0





For A* choose minimal $f(n) = g(n) + h(n)$

Heuristic Properties: A Roadmap



A* is only as good as its heuristic. But what makes a “good” heuristic?

We consider three key properties of heuristics:

Dominance → Helps us choose between multiple admissible heuristics

Given several valid heuristics, which one performs best?

Admissibility → Ensures A* finds optimal solutions

Without this, A* may return suboptimal paths

Consistency → Enables more efficient search

Allows A* to avoid redundant work

Goal: Design / select heuristics that make A* both optimal and efficient.

Admissible (Global) Heuristics

Under what conditions is A* guaranteed to find optimal solutions?

Definition. Heuristic $h(n)$ is admissible if:

$0 \leq h(n) \leq h^*(n)$ for all n where $h^*(n)$ is the true cost from n to goal.

Interpretation: $h(n)$ is admissible if it never overestimates the cost to reach the destination node.

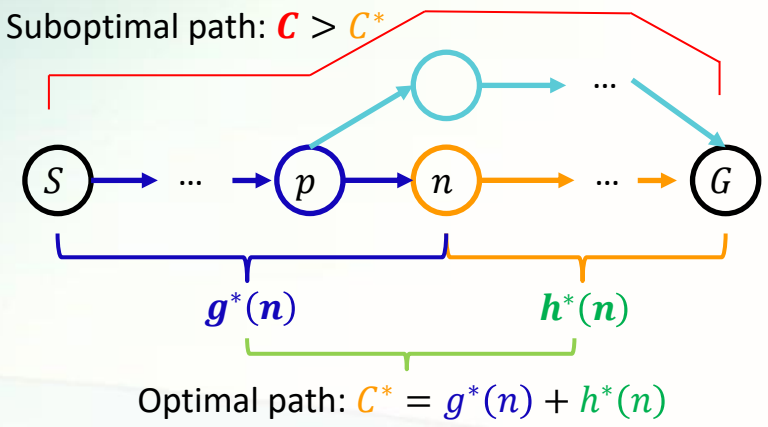
Examples:

Straight-line distance for route finding
Manhattan distance for grid worlds
Number of misplaced tiles for 8-puzzle
Minimum Spanning Tree in the Traveling Salesperson problem.
Number of attacking pairs: in the 8-Queens problem.

Examples?

- ✗ Max(horizontal distance, vertical distance)
- ✗ Random numbers
- ✗ Actual driving time (overestimates driving distance)

A* Proof of Optimality



Theorem. A*, with admissible heuristic, will find an optimal solution.

Proof by Contradiction.

Suppose the optimal path has cost C^*, but A* returns a suboptimal path with cost $C > C^*$.

Then some first node n on the optimal path remains unexpanded with $f(n) \geq C > C^*$ (otherwise A* would have expanded n before the suboptimal goal.

Let $g^*(n)$ be the cost of the optimal path from start to n and $h^*(n)$ be the cost of the optimal path from n to the goal.

Recall, $f(n)$ is the path cost through n to a goal (and how A* decides which node to expand next).

$$
\begin{aligned}
f(n) &= g(n) + h(n) \\
&= g^*(n) + h(n) && \text{Since } n \text{ is on an optimal path} \\
&\leq g^*(n) + h^*(n) && \text{Admissability: } h(n) \leq h^*(n) \\
&= C^* && \text{By definition, } C^* = g^*(n) + h^*(n)
\end{aligned}
$$

However, our assumption was $f(n) > C^*$.
Contradiction. □

Full A* Algorithm

Put the start state S on the priority queue called OPEN.

Repeat until OPEN is empty,

States already expanded

Remove from OPEN and add to CLOSED a node n for which $f(n) = g(n) + h(n)$ is minimum.

If n is a goal node

Exit (recover path by tracing back pointers from n to S)

Expand n, generating all successors and attach to pointers back to n.

For each successor n' of n:

If n' is not already on OPEN or CLOSED: # New node found

$h(n') = compute(n', goal)$

$g(n') = g(n) + c(n, n')$

$f(n') = g(n') + h(n')$

Place n' on OPEN.

Else If n' is already on OPEN or CLOSED:

If $g(n')$ is lower for the new version of n'.

Redirect pointers backward from n' along path yielding lower $g(n')$.

If (n' is already on OPEN) then update n' on OPEN;

else add n' to OPEN

Exit with failure

Heuristic Dominance

We have considered how different algorithms trade off optimality, completeness, and efficiency. But how can we evaluate the effectiveness of our heuristic?

That is, if we are using A*, how do we choose between different admissible heuristics?

Definition. Heuristic h_2 is said to ***dominate*** heuristic h_1 if both h_1 and h_2 are admissible and for all nodes n:

$$h_1(n) \leq h_2(n)$$

Recall, admissible heuristics are always optimistic (underestimate true cost):

$$h_1(n) \leq h_2(n) \leq h_3(n) \leq \cdots \leq h^*(n)$$

where $h^*(n)$ is the true optimal cost to goal.

→ a dominating heuristic gives us estimates closer to the real costs (that are still admissible).

Having dominant heuristics is important!

More accurate estimates → better guidance → fewer wrong paths explored

Heuristic Dominance: Example

8-puzzle: may only slide tiles into the empty space (no jumping or diagonal moves).

Challenge: Find the minimum sequence of moves to reach the goal state from any given starting configuration.

3	1	2
7	5	6
8	4	

Example State: n

1	2	3
4	5	6
7	8	

Goal State

h_3 provides the tightest estimate ($6 \leq 9 \leq 12$), meaning A* with h_3 would expand fewer nodes and reach the goal more efficiently than with h_2 or h_1.

h_1: (Misplaced tiles): Count tiles not in goal position

$$h_1(n) = 1 + 1 + 1 + 1 + 0 + 0 + 1 + 1 = 6$$

h_2: (Manhattan distance): Sum of distances each tile must move

$$h_2(n) = 1 + 1 + 2 + 1 + 0 + 0 + 1 + 2 = 8$$

h_3: (Manhattan + Linear Conflict):
Manhattan distance $+ 2 \cdot$ number of linear conflicts

Two tiles are in *linear conflict* if:

- They are in the correct row/column.
- A pairwise comparison indicates they are not in correct relative order.
- Each conflict adds 2 to the heuristic indicating the minimum moves needed to resolve the conflict.

$$\begin{aligned} h_3(n) &= 8 + 2 \cdot ([row_1] + [row_2] + [row_3] + [col_1] + [col_2] + [col_3]) \\ &= 8 + 2 \cdot ([1 + 1 + 0] + [0 + 0 + 0] + [0 + 0] + [0] + [0] + [0]) \\ &= 12 \end{aligned}$$

(Observe no column conflicts here: multiple tiles in each column are in the goal column but out of order.)

From Admissible to Consistent

Admissibility A global promise about the goal

Never overestimate the cost to reach the goal.

But this says nothing about the journey along the way...

What if we made a promise about every single step?

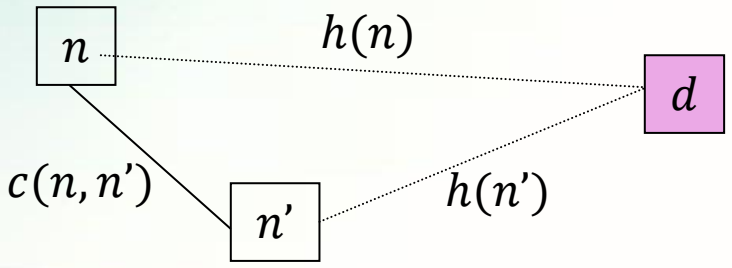
Consistency

Consistency (Local Optimism)

A graph-search (without reopening) generates an optimal solution if $h(n)$ is a consistent:

- $h(n)$ is **consistent** (or monotonic) if for every node n and for every successor node n' of n:

$$h(n) \leq c(n, n') + h(n')$$



(This is a form of triangle inequality.)

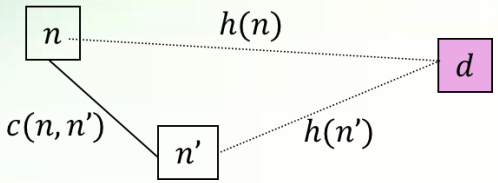
Example. Euclidean distance heuristic.
It represents the shortest possible path between two points, so the direct distance to the goal is always less than or equal to any indirect route through neighboring nodes.

$h(n)$ is consistent → $h(n)$ is admissible

With a consistent heuristic, the first time we reach a state, it will be on the optimal path. So we never have to re-add a state to the frontier (or check the CLOSED list).

(An inconsistent heuristic may end up with multiple paths reaching the same state.)

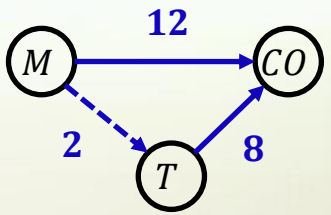
Inconsistent Example



$$h(n) \leq c(n, n') + h(n')$$

Consider a situation where your GPS estimates distance to destination based on old road information.

Now imagine you are at Furman main entrance with 12 miles using surface roads to your destination: Cook Out.



You drive 2 miles to reach Timmons gate. From the Timmons gate, GPS discovered a more direct highway access that significantly shortens the remaining distance to Cook Out (only 8 miles left).

Do we have consistency? $12 \overset{?}{\leq} 2 + 8$ **No.**

A* Optimality

A* Version	Requirement for Optimality	Notes
Tree-search	Admissible heuristic	No **CLOSED** list—can re-expand states.
Graph-search with reopening	Admissible heuristic	Checks **CLOSED** nodes for cheaper paths.
Graph-search without reopening	Consistent heuristic	Skips **CLOSED** nodes entirely.

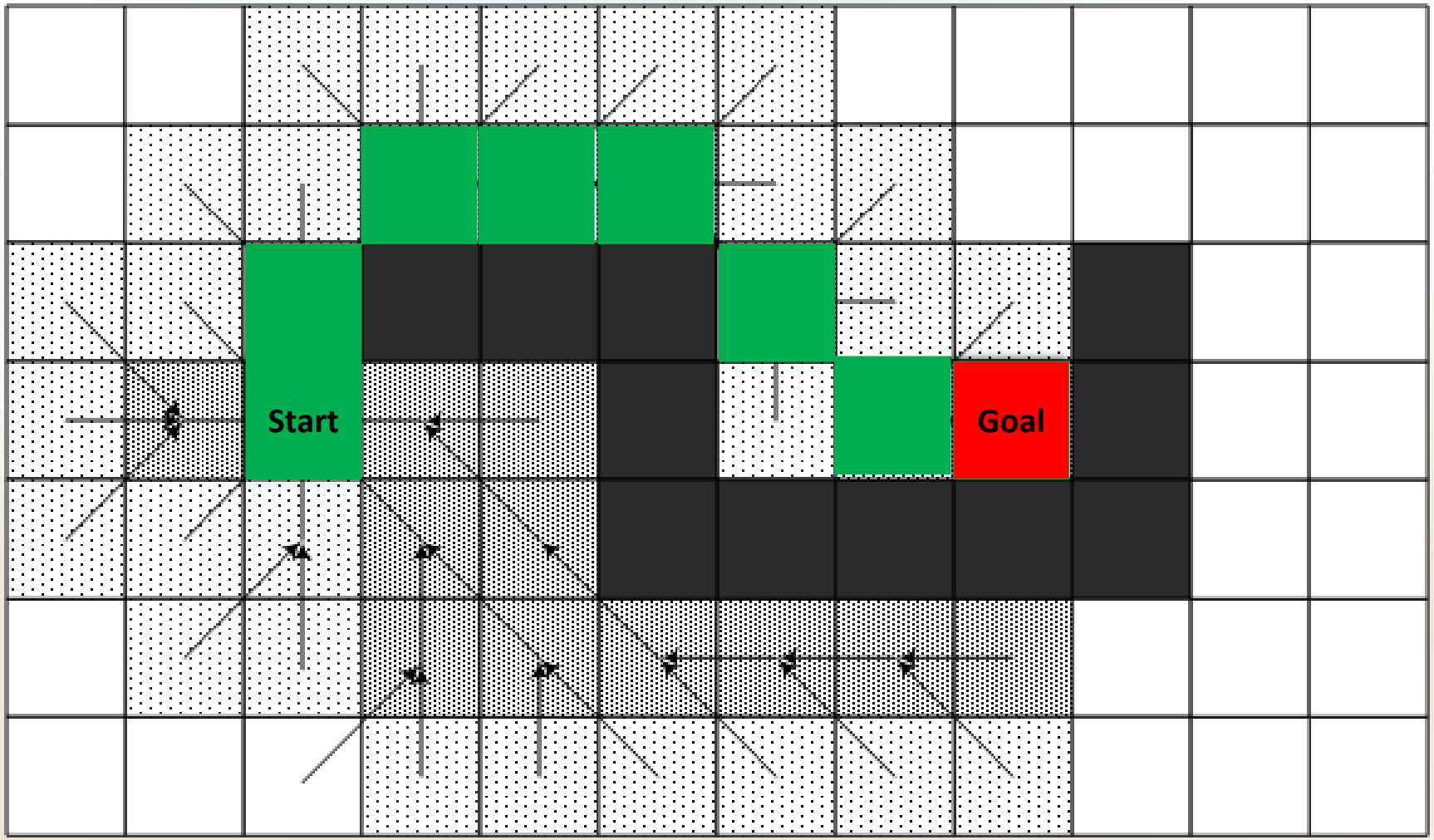
Tree-search has no **CLOSED** list, allowing the same state to be expanded multiple times via different paths. Inefficient, but guarantees finding the optimal solution with any admissible heuristic.

Graph-search with reopening maintains **CLOSED** but checks it for cheaper paths. If found, it updates costs and re-opens the node. This safety net means admissibility alone suffices.

Graph-search without reopening skips **CLOSED** nodes entirely, assuming the first path found is optimal. This is only true with a consistent heuristic, which ensures f-values never decrease along any path.

Bottom line: Consistency enables efficiency; no reopening is needed when you are guaranteed the first path is best.

A* Avoids the Best-First Trap



Brief A* Analysis

Time complexity: $O(b^d)$ where:

- b is the branching factor and
- d the depth of the optimal solution.

However, with a good heuristic, A* can, in practice, be $O(b^{\epsilon \cdot d})$ where $0 < \epsilon < 1$ represents how much the heuristic can reduce the search space.

Space complexity: $O(b^d)$

A* consumes lots of memory: $O(\#states)$ (**OPEN**, **CLOSED**, back-pointers). Thus, may run out on large problems.

If the heuristic can be consistent, we earn space efficiency by not re-adding nodes to the open list (by re-opening **CLOSED** nodes).

IDA*

We have encountered the problem of running out of memory before. Our solution to meld BFS and DFS was iterative deepening.

IDA*: incrementally increasing the maximum f-value $f(n) = g(n) + h(n)$

Time complexity: $O(b^d)$

where:

- b is the branching factor and
- d the depth of the optimal solution.

Space complexity: $O(bd)$

stores path like DFS, but re-explores.

Complete? ✓

with an admissible heuristic.

Optimal? ✓

(Just as A*)

Beam Search

A* is optimal but uses exponential memory. IDA* is memory efficient but may re-explore nodes. What if we need something in between?

Motivation: Reduce memory usage by ***limiting frontier size*** while allowing our heuristic to guide us.

Local Beam Search

- The frontier is a priority queue with fixed size beam width k.
- Expand all k nodes simultaneously
- Keep best k successors for next iteration

Complete? ✗

Optimal? ✗

Can discard (optimal) paths by keeping only locally promising nodes

Space? $O(k)$

Time? $O(k \cdot d)$

Stochastic Beam Search

- Beam width k
- Selection of k successors is probabilistic.
  - All successors are assigned probabilities based on 'goodness'.
  - k are then selected according to that distribution.
- Helps avoid getting trapped in local minima by introducing randomness into node selection

Beyond Optimal Search: Weighted A*

What if optimality is not worth the computational cost? Sometimes “good enough” solutions found quickly are better than optimal solutions found slowly.

Weighted A*: $f(n) = g(n) + w \cdot h(n)$ where $w > 1$

Weight w is a tunable parameter that can emphasize the heuristic more heavily:

- $w = 1$: balanced between actual cost $g(n)$ and heuristic $h(n)$ — Standard A*
- $w \to \infty$: making search more "optimistic" about reaching the goal by emphasizing $h(n)$ — Greedy Best-First

Complete?	✓	Best-First and A* are complete; we are still using an admissible heuristic.
Optimal?	✗	Solution cost is guaranteed to be $\leq w \cdot C^*$ where C* is optimal cost
Space?	$O(b^d)$	Often expands significantly fewer nodes in practice as w increases.
Time?	$O(b^d)$	Often faster than A* in practice.

Used in real-time systems, video games, robotics.

Beyond Optimal Search: Anytime Algorithms

What if we do not know how much time we have to search?

Common in game environments due to issues with frame rate.

Idea:

- **Start** with a quick, possibly suboptimal solution
- **Improve** solution quality over time if more time is available
- **Stop** anytime and return current best solution
- **Graceful** degradation: longer time → better solutions

Goal: Always have *some* answer ready.

Anytime A* Approaches

- With Weighted A*, start with high w, gradually reduce
- **Iterative deepening** with increasing time/space limits
- For **beam search**, increase beam width over time

Properties

- **Interruptible**: Can stop and return solution at any time
- **Monotonic improvement**: Solution quality never decreases
- **Diminishing returns**: Biggest improvements happen early

Games, online planning, real-time decision making

Practical Consideration: When To Use?

Algorithm	When to Use
Greedy Best-First	• Quick approximate solutions • Good heuristic available • Optimality not important
A*	• Need optimal solution • Have good admissible heuristic • Memory is available
IDA*	• Need optimal solution • Memory is limited • Heuristic is good (low branching factor)
Beam Search	• Large search spaces • Optimal solution not required • Time/memory constraints
Anytime Algorithms	• Uncertain time constraints • Need solution quickly, improve if time allows • Real-time systems (games, robotics)